

AeroBenchVPython 분석 및 AI Combat 통합 가이드

작성일: 2026-04-21

분석 대상: [external_repo/AeroBenchVPython](#)

목적: AI Combat 플랫폼 활용 가능 영역 식별

1. 저장소 개요

AeroBenchVPython은 **Air Force Research Laboratory**가 신경망 기반 비행 제어 시스템의 형식 검증(Formal Verification & Validation)을 위해 공개한 **순수 Python F-16 비행 시뮬레이터**다.

- 원 저자: Stanley Bak (AFRL)
- 기반 모델: Stevens & Lewis의 "Aircraft Control and Simulation" (F-16)
- 라이선스: MIT
- 특징: JSBSim 불필요, 외부 의존성 최소화 (scipy, numpy만 필요)

2. 저장소 구조

```
AeroBenchVPython/  
├── code/aerobench/  
│   ├── run_f16_sim.py      메인 시뮬레이션 루프 (RK45 수치 적분)  
│   ├── util.py            StateIndex 열거형, 유틸리티  
│   └── lowlevel/  
│       ├── subf16_model.py  13차 비선형 ODE (핵심)  
│       ├── morellif16.py    Morelli 다항식 공기역학 모델  
│       ├── low_level_controller.py LQR 피드백 제어기  
│       ├── adc.py           대기 데이터 (마하수, 동압)  
│       ├── thrust.py        추진력 룩업테이블  
│       ├── cx/cy/cz/cl/cm/cn.py 공기역학 계수 (이중선형 보간)  
│       └── dlda/dldr/dnda/dndr.py 제어면 효율 계수  
│   ├── highlevel/  
│       ├── autopilot.py     하이브리드 오토마톤 베이스 클래스  
│       └── controlled_f16.py LQR + F-16 ODE 통합 래퍼  
│   ├── examples/  
│       ├── gcas/            지상충돌방지 시스템 (GCAS)  
│       ├── waypoint/        경로점 추종 자동조종  
│       └── acasxu/           신경망 기반 충돌회피 (ONNX)  
│   └── visualize/           2D/3D 플롯, 애니메이션
```

3. 핵심 물리 모델

3.1 F-16 13차 상태 벡터

인덱스	변수	설명	단위
0	VT	기속도	ft/s
1	ALPHA	받음각	rad
2	BETA	옆미끄럼각	rad
3	PHI	롤각	rad
4	THETA	피치각	rad
5	PSI	요각 (헤딩)	rad
6	P	롤각속도	rad/s
7	Q	피치각속도	rad/s
8	R	요각속도	rad/s
9	POS_N	북쪽 변위	ft
10	POS_E	동쪽 변위	ft
11	ALT	고도	ft
12	POW	엔진 파워 상태	0-10

3.2 제어 입력 (4개)

변수	범위	설명
Throttle	0 - 1	스로틀
Elevator	$\pm 25^\circ$	엘레베이터
Aileron	$\pm 21.5^\circ$	에일러론
Rudder	$\pm 30^\circ$	러더

3.3 고수준 명령 인터페이스 (LQR 입력)

LQR 제어를 통해 저수준 제어면 대신 고수준 명령으로 비행 가능:

```
u_ref = [Nz_ref, ps_ref, Ny_r_ref, throttle]
Nz:   법선 가속도 (G), 범위 [-1, 6]
ps:   안정성 롤각속도 (rad/s)
Ny_r: 옆 G + 요각속도 (rad/s)
```

3.4 LQR 제어기 구조

```
u_ref (Nz, ps, Ny_r, throttle)
|
```

```

▼
LowLevelController (low_level_controller.py)
├─ K_long: 1×3 이득 행렬 (세로 제어)
├─ K_lat: 2×5 이득 행렬 (가로 제어)
├─ 적분기 3개 (Nz, ps, Ny_r 오차 누적)
├─ 평형점 (xequil, uequil) 추가
└─
  │
  ▼
u_deg [throttle, elevator, aileron, rudder]
  │
  ▼
subf16_model (F-16 ODE) → x_dot

```

평형 조건 (레벨 비행 기준점):

```

xequil = [502.0 ft/s, 0.0389 rad, 0, 0, 0.0389 rad, 0, 0, 0, 0, 0, 0, 1000 ft,
9.0567]
uequil = [0.1395, -0.7496 deg, 0, 0]

```

4. 자동조종 예제 분석

4.1 GCAS (Ground Collision Avoidance System)

파일: `examples/gcas/gcas_autopilot.py`

하이브리드 상태기계 (4개 모드):

```

standby
├─ alt < 1000 ft AND 기수가 지면을 향할 때
└─
  │
  ▼
roll ─ (날개 수평 AND 롤각속도 낮음) ─► pull
  ▲                                     │
  │                                     │
  └─ (기수 충분히 올라감) ───────────┘
                                     │
                                     ▼
                                     standby

```

핵심 조건 판별 함수:

```

# 날개 수평 여부
are_wings_level(x_f16):
    return |phi mod 2π| < 5°

# 롤각속도 낮음 여부
is_roll_rate_low(x_f16):
    return |p| < 10°/s

# 기수가 충분히 올라갔는가

```

```

is_nose_high_enough(x_f16):
    return (theta - alpha) > 0 # flight path angle > 0

# Hard Deck 이하인가
is_above_flight_deck(x_f16):
    return alt >= 1000 ft

```

각 모드의 제어 명령:

모드	Nz	ps	설명
standby	0	0	대기
roll	0	PD(phi, p)	날개 수평화
pull	5	0	기수 올리기 (5G)
waiting	0	0	2초 대기 후 roll

4.2 Waypoint 추종 자동조종

파일: [examples/waypoint/waypoint_autopilot.py](#)

제어 루프 계층:

```

목표 경로점 (N, E, Alt)
├── track_airspeed()    throttle = Kv × (Vt_cmd - Vt)
├── track_altitude()    Nz = Kalt × h_err - Khdot × h_dot + Nz_bank
└── heading tracking
    ├── get_phi_to_track_heading() phi_cmd = Kp_psi×psi_err - Kd_psi×r
    └── track_roll_angle()         ps = Kp_phi×phi_err - Kd_phi×p

```

검증된 제어 이득:

제어 루프	이득	값
속도 비례	Kv	0.25
고도 비례	Kalt	0.005
고도율 미분	Khdot	0.02
헤딩 비례	Kp_psi	5.0
헤딩 미분	Kd_psi	0.5
롤 비례	Kp_phi	0.75
롤 미분	Kd_phi	0.5
최대 뱅크각	-	65°

제어 루프	이득	값
목표 속도	-	550 ft/s

4.3 ACASXU (신경망 충돌회피)

파일: `examples/acasxu/acasxu_autopilot.py`

- ONNX 기반 신경망 추론 (45개 네트워크, 수직/수평 위험도 조합)
- 5개 이산 명령: `clear`, `weak-left`, `weak-right`, `strong-left`, `strong-right`
- 다중 항공기 동시 처리 지원

5. AI Combat 플랫폼과의 비교

항목	AI Combat (JSBSim 기반)	AeroBenchVVPython
물리 엔진	JSBSim C++ (Python 바인딩)	순수 Python (scipy RK45)
F-16 모델	JSBSim XML 설정	Stevens & Lewis 록업테이블
액션 공간	이산 5×9×5 = 225가지	연속 (Nz, ps, Ny_r, throttle)
AI 프레임워크	py_trees 행동트리 (YAML)	Autopilot 하이브리드 오토마톤
좌표계	geodetic + NEU (m 단위)	Flat Earth NED (ft 단위)
다중 항공기	1v1 (SingleCombatEnv)	지원 (acasxu multi)
시각화	Tacview ACMI	matplotlib 2D/3D
검증 목적	경진대회 플랫폼	형식 검증 (V&V)

6. 활용 가능 영역 (우선순위)

6-A. GCAS 알고리즘 → Hard Deck 안전 BT 노드 ★★★

배경: AI Combat에서 고도 < 1,000 ft 시 즉시 패배(Hard Deck 위반).

활용: GCAS 조건 함수들을 BT Condition 노드로 직접 포팅.

구현 대상 노드:

```
# conditions.py에 추가 가능한 노드들
class IsNoseHighEnough(BaseCondition):
    """theta - alpha > 0 (flight path angle > 0)"""

class IsWingsLevel(BaseCondition):
    """|phi mod 2π| < cfg_eps_phi (기본 5°)"""

class IsRollRateLow(BaseCondition):
    """|p| < cfg_eps_p (기본 10°/s)"""

class IsHardDeckDanger(BaseCondition):
```

```

    """alt < 1500 ft AND flight path angle < 0"""

# actions.py에 추가 가능한 노드
class GcasPullUp(BaseAction):
    """GCAS pull up: 고도 올리기 (급상승)"""

class GcasRollLevel(BaseAction):
    """GCAS roll: 날개 수평화"""

```

JSBSim 데이터 매핑 (Catalog 속성):

```

phi    = obs["attitude/phi-rad"]      # 롤각
theta  = obs["attitude/theta-rad"]    # 피치각
alpha  = obs["aero/alpha-rad"]        # 받음각
p       = obs["velocities/p-rad_sec"]  # 롤각속도
alt     = obs["position/h-sl-ft"]      # 고도 (ft)

```

6-B. Waypoint 제어 이득 → 기동 BT 노드 정밀화 ★★★

배경: AI Combat BT 액션 노드의 헤딩/고도 추종 로직에 임의 이득 사용 중.

활용: AeroBench의 검증된 F-16 제어 이득을 참조값으로 적용.

이산 액션 변환 로직 예시:

```

def heading_err_to_action(psi_err_deg: float) -> int:
    """헤딩 오차 → delta_heading_idx (HeadingIdx)"""
    # AeroBench Waypoint 이득 기반 임계값
    if psi_err_deg < -45: return HeadingIdx.HARD_LEFT      # 0
    elif psi_err_deg < -20: return HeadingIdx.STRONG_LEFT # 1
    elif psi_err_deg < -8:  return HeadingIdx.MED_LEFT    # 2
    elif psi_err_deg < -2:  return HeadingIdx.SOFT_LEFT   # 3
    elif psi_err_deg <= 2:  return HeadingIdx.STRAIGHT    # 4
    elif psi_err_deg <= 8:  return HeadingIdx.SOFT_RIGHT  # 5
    elif psi_err_deg <= 20: return HeadingIdx.MED_RIGHT   # 6
    elif psi_err_deg <= 45: return HeadingIdx.STRONG_RIGHT # 7
    else:                  return HeadingIdx.HARD_RIGHT   # 8

def alt_err_to_action(alt_err_ft: float) -> int:
    """고도 오차 → delta_altitude_idx (AltitudeIdx)"""
    if alt_err_ft < -500: return AltitudeIdx.DIVE          # 0
    elif alt_err_ft < -100: return AltitudeIdx.DESCEND     # 1
    elif alt_err_ft <= 100: return AltitudeIdx.MAINTAIN    # 2
    elif alt_err_ft <= 500: return AltitudeIdx.CLIMB       # 3
    else:               return AltitudeIdx.CLIMB_FAST     # 4

```

6-C. 빠른 RL 사전학습 환경 ★★★

배경: JSBSim은 XML 로딩 + C++ 프로세스 통신 오버헤드로 RL 학습 속도 제한.

활용: AeroBench 순수 Python 시뮬레이터로 빠른 롤아웃 생성 → JSBSim 파인튜닝.

아키텍처:

```
Phase 1: AeroBench Gym 환경
├─ run_f16_sim() 기반 1v1 전투 시뮬레이션
├─ 보상: ATA 각도, 거리, 고도, HP 기반
├─ 빠른 배치 학습 (JSBSim 대비 10~50x)
│   │
│   ▼ 사전학습된 정책 전달
Phase 2: JSBSim 파인튜닝
├─ SingleCombatEnv (기존 환경)
├─ 정밀 물리 모델로 최종 조정
```

구현 스킴레톤 ([ai-combat-sdk/src/simulation/envs/AeroBench/](#)):

```
class AeroBenchCombatEnv(gymnasium.Env):
    """AeroBench 기반 빠른 RL 사전학습 환경 (1v1)"""

    observation_space = ... # AeroBench 13차 상태 × 2 항공기
    action_space = spaces.Discrete(225) # 5×9×5 (AI Combat 호환)

    def reset(self):
        # 랜덤 초기 조건 (F-16 평형 근방)
        ...

    def step(self, action):
        # 이산 액션 → u_ref 변환 → controlled_f16() 1스텝
        # 보상: ATA 각도, 거리, Hard Deck, 에너지
        ...
```

6-D. 상태 벡터 매퍼 유틸리티 ★★

AeroBench 알고리즘을 JSBSim 데이터로 실행하기 위한 변환 클래스.

```
# src/control/aerobench_adapter.py (신규 파일)

FT_PER_METER = 3.28084
FT_PER_S_PER_M_PER_S = 3.28084

class AeroBenchStateAdapter:
    """JSBSim Catalog 속성 → AeroBench StateIndex 변환"""

    @staticmethod
    def from_jsbsim(obs: dict) -> np.ndarray:
        """JSBSim observation dict → AeroBench x_f16 (13차 벡터)"""
```

```

x = np.zeros(13)
x[0] = obs["velocities/vt-fps"] # VT: ft/s (단위 동일)
x[1] = obs["aero/alpha-rad"] # ALPHA
x[2] = obs["aero/beta-rad"] # BETA
x[3] = obs["attitude/phi-rad"] # PHI
x[4] = obs["attitude/theta-rad"] # THETA
x[5] = obs["attitude/psi-rad"] # PSI
x[6] = obs["velocities/p-rad_sec"] # P
x[7] = obs["velocities/q-rad_sec"] # Q
x[8] = obs["velocities/r-rad_sec"] # R
x[9] = obs["position/distance-from-start-lat-mt"] * FT_PER_METER # POS_N
x[10] = obs["position/distance-from-start-lon-mt"] * FT_PER_METER # POS_E
x[11] = obs["position/h-sl-ft"] # ALT: ft (단위 동일)
x[12] = obs["propulsion/engine/thrust-lbs"] / 1000 # POW: 근사
return x

```

6-E. ACASXU → Red Team AI 충돌회피 베이스라인 ★★

활용: 기본 제공 Red Team 에이전트에 신경망 기반 기동회피 탑재.

ACASXU 명령 → AI Combat 이산 액션 변환:

```

clear      → STRAIGHT + MAINTAIN (HeadingIdx.4, AltitudeIdx.2)
weak-left  → SOFT_LEFT (HeadingIdx.3)
weak-right → SOFT_RIGHT (HeadingIdx.5)
strong-left → HARD_LEFT + CLIMB (HeadingIdx.0, AltitudeIdx.3)
strong-right → HARD_RIGHT + CLIMB (HeadingIdx.8, AltitudeIdx.3)

```

6-F. JSBSim 모델 검증 도구 ★

활용: 동일 초기조건에서 AeroBench와 JSBSim의 궤적을 비교하여 JSBSim F-16 설정 검증.

```

# tools/validate_jsbsim.py (신규)
# 1. AeroBench로 기준 궤적 생성
# 2. JSBSim에서 동일 초기조건 실행
# 3. 위치/자세 오차 플롯

```

7. 구현 로드맵

우선순위	작업	관련 파일	예상 공수
1	GCAS 조건 BT 노드 4개 포팅	src/behavior_tree/nodes/conditions.py	0.5일
2	GCAS 액션 BT 노드 2개 포팅	src/behavior_tree/nodes/actions.py	0.5일
3	이산 액션 ↔ 연속 명령 변환 유틸	src/control/aerobench_adapter.py	1일

우선순위	작업	관련 파일	예상 공수
4	AeroBench Gym 환경 래퍼	src/simulation/envs/AeroBench/	3일
5	JSBSim 검증 스크립트	tools/validate_jsbsim.py	1일
6	ACASXU Red Team 통합	redteams/acasxu_agent/	2일

8. 기술적 주의사항

단위 변환

물리량	AI Combat (JSBSim)	AeroBenchVVPython
거리	m	ft (× 3.28084)
속도	m/s	ft/s (× 3.28084)
각도	rad	rad (동일)
고도	m	ft (× 3.28084)

좌표계 차이

- **AeroBench:** Flat Earth NED (North-East-Down), ft 단위, 원점 고정
- **JSBSim:** geodetic (위도/경도/고도) → NEU 변환, m 단위
- 변환 함수: [src/simulation/envs/JSBSim/utils/utils.py](#)의 `LLA2NEU`, `NEU2LLA` 활용

F-16 모델 차이

AeroBench와 JSBSim은 동일한 F-16을 모델링하지만 구현 방식이 다름:

- **AeroBench:** Stevens & Lewis 텍스트북 기반 단순화 (검증 목적)
- **JSBSim:** 더 상세한 XML 기반 설정, 추가 서브시스템 포함

정확한 수치 매칭은 어렵지만 정성적 거동은 유사하므로 알고리즘 로직 이식에는 문제없음.

9. 참고 자료

- 원 저장소: [external_repo/AeroBenchVVPython/](#)
- 기반 교재: Stevens & Lewis, "Aircraft Control and Simulation", 2nd Ed.
- 관련 논문: Morelli, "Global Nonlinear Parametric Modelling with Application to F-16 Aerodynamics"
- ACASXU 논문: Kochenderfer et al., "Optimized Airborne Collision Avoidance"
- 검증 논문: Bak et al., "Verifying the Safety of Autonomous Systems with Neural Network Controllers"